

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



THỰC TẬP CƠ SỞ

Weekly Report

Đề tài: XÂY DỰNG HỆ THỐNG GỢI Ý SẢN PHẨM

Giai đoạn 3: xây dựng mô hình

Giảng viên	: Kim Ngọc Bách
Họ và tên sinh viên	: Nguyễn Văn Trường
Mã sinh viên	: B23DCCN870
Lớp	: D23CQCN02-B
Nhóm	: 11

Hà Nội – 2026

Nội dung

1. TỔNG QUAN HỆ THỐNG	3
1.1. Đặt tả Bài toán & Mục tiêu	3
1.2. Bản đồ Công nghệ (Technology Stack)	3
1.3. Kiến trúc Tổng thể Hệ thống	4
2. ĐƯỜNG ống XỬ LÝ DỮ LIỆU (DATA PIPELINE)	5
2.1. Bước 1: Nạp Dữ liệu (load_data)	6
2.2. Bước 2: Phân chia Dữ liệu Trực giao theo Thời gian (Temporal Split)	6
2.3. Bước 3: Khởi tạo Ma Trận Tương tác (build_matrix)	6
3. CHI TIẾT THUẬT TOÁN KHUYẾN NGHỊ	7
3.1. Mô hình 1: Lọc cộng tác dựa trên Người dùng (User-Based Collaborative Filtering)	7
3.2. Mô hình 2: Lọc cộng tác dựa trên Mục tiêu (Item-Based Collaborative Filtering) ...	8
3.3. Mô hình 3: Phân rã Ma Trận (Matrix Factorization - Funk SVD)	8
4. KHUNG ĐÁNH GIÁ MÔ HÌNH (EVALUATION FRAMEWORK)	10
4.1. Chỉ số sai số hồi quy (Regression Metrics)	10
4.2. Chỉ số xếp hạng danh sách (Ranking Metrics)	10
5. THIẾT KẾ KIẾN TRÚC & LƯỒNG DỮ LIỆU ĐỘNG	11
5.1. Mô hình Facade & Quản lý Singleton (models.py & app.py)	11
5.2. Lồng xử lý yêu cầu API (FastAPI REST Endpoints)	11
6. ĐÁNH GIÁ TOÀN DIỆN & PHÂN TÍCH RỦI RO KỸ THUẬT	12
6.1. Các ưu điểm nổi bật trong thiết kế của CineRec	12
6.2. Các nhược điểm cố hữu và rủi ro vận hành	12
7. LỘ TRÌNH PHÁT TRIỂN & ĐỀ XUẤT TỐI ƯU HÓA	14
7.1. Các cải tiến ngắn hạn (Quick Wins)	14
7.2. Các cải tiến trung hạn (Algorithmic Upgrades)	15
7.3. Thiết kế tầm nhìn dài hạn (Production-Ready Architecture)	15
8. KẾT LUẬN & KỶ VỌNG ĐẠT ĐƯỢC	15

1. TỔNG QUAN HỆ THỐNG

1.1. Đặt tả Bài toán & Mục tiêu

CineRec là một hệ thống khuyến nghị phim (Recommendation System) được nghiên cứu và phát triển dựa trên tập dữ liệu chuẩn thức **MovieLens ml-100k** được cung cấp bởi GroupLens Research.

Mục tiêu cốt lõi của hệ thống là giải quyết bài toán **Collaborative Filtering** (Lọc cộng tác): dự đoán mức độ ưa thích (thông qua điểm số đánh giá - *rating*) của một người dùng (*user*) đối với một bộ phim (*movie/item*) mà họ chưa từng tương tác trong quá khứ. Dựa trên các điểm số dự đoán này, hệ thống sẽ tiến hành xếp hạng và đề xuất danh sách Top- N tác phẩm điện ảnh tối ưu nhất cho từng cá nhân.

Bảng thông số cơ bản của tập dữ liệu MovieLens ml-100k:

Thuộc tính	Giá trị	Ý nghĩa kỹ thuật
Số lượng người dùng (U)	943	Số lượng hàng trong ma trận tương tác đầy đủ
Số lượng phim (I)	1.682	Số lượng cột trong ma trận tương tác đầy đủ
Số lượng ratings (R)	100.000	Tổng số lượng mẫu dữ liệu đã gán nhãn
Thang điểm tương tác	1 $\rightarrow$	Thang đo explicit feedback (liên tục/số nguyên)
Độ thưa (Sparsity)	$\approx$	Tỷ lệ dữ liệu khuyết thiếu: $1 - \frac{R}{U \times I}$

1.2. Bản đồ Công nghệ (Technology Stack)

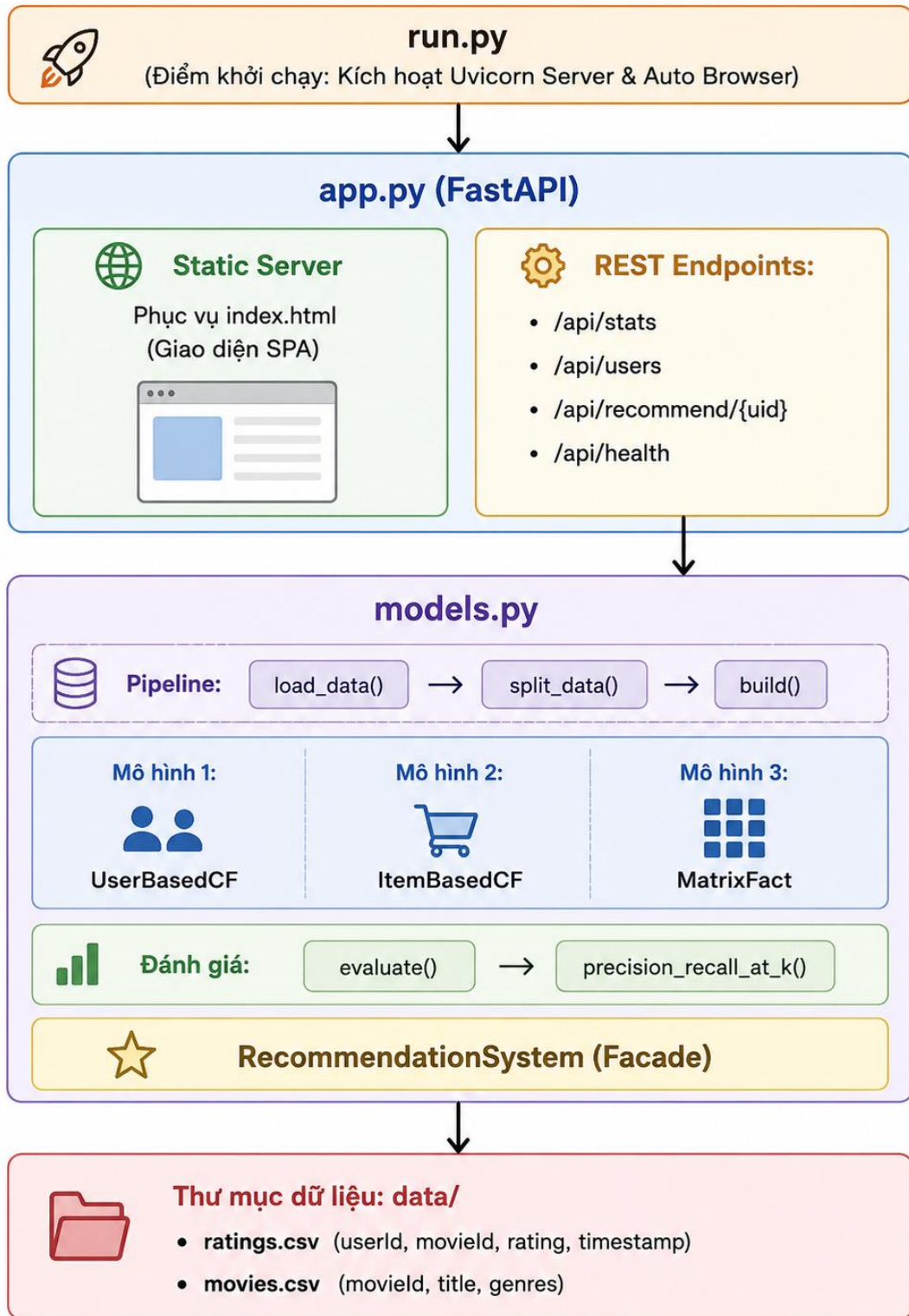
Hệ thống được thiết kế theo kiến trúc module hóa phân tầng, đảm bảo tính độc lập giữa lõi xử lý học máy, dịch vụ backend và giao diện người dùng:

- **Lớp Machine Learning Core (ML Core):**
 - NumPy & Pandas: Đảm nhiệm tính toán đại số tuyến tính, nhân ma trận hiệu năng cao và xử lý cấu trúc dữ liệu bảng.
 - scikit-learn: Trích xuất các hàm đánh giá sai số tiêu chuẩn như MAE (Mean Absolute Error) và MSE (Mean Squared Error).

- **Lớp Backend API:**
 - FastAPI: Web framework bất đồng bộ (Asynchronous) hiệu năng cực cao, hỗ trợ tự động sinh tài liệu OpenAPIs (Swagger).
 - Uvicorn: ASGI web server giúp vận hành ứng dụng tối ưu trên môi trường đơn luồng/đa luồng.
- **Lớp Frontend (Giao diện Web):**
 - Vanilla JS & HTML5/CSS3: Ứng dụng mô hình Single Page Application (SPA) thuần túy không phụ thuộc thư viện ngoài, tối ưu hóa tốc độ tải và khả năng tương thích.
 - Canvas 2D API: Vẽ biểu đồ động đường cong hội tụ (Loss Curve) trực tiếp trên trình duyệt.
- **Lớp Terminal Interface:**
 - Python Standard Library: Bộ thư viện chuẩn của Python giúp triển khai công cụ chạy thử nghiệm dòng lệnh (demo_console.py).

1.3. Kiến trúc Tổng thể Hệ thống

Luồng xử lý từ điểm kích hoạt hệ thống, nạp dữ liệu, huấn luyện ngoại tuyến (offline training), đến việc cung cấp dịch vụ thông qua RESTful API được mô tả trực quan qua sơ đồ dưới đây:



2. ĐƯỜNG ống XỬ LÝ DỮ LIỆU (DATA PIPELINE)

Trước khi đưa vào mô hình hóa, dữ liệu thô trải qua đường ống xử lý nghiêm ngặt nhằm đảm bảo tính thực tế và ngăn ngừa các lỗi rò rỉ thông tin (data leakage).

[ratings.csv] —► [Sắp xếp theo Timestamp] —► [Temporal Split 80/20] —► [Build Sparse Matrix R]

2.1. Bước 1: Nạp Dữ liệu (load_data)

Hệ thống tiến hành đọc các file dữ liệu nguồn .csv, xử lý định dạng và xây dựng một bảng băm ánh xạ (lookup dictionary) để tra cứu siêu dữ liệu (metadata) của phim trong độ phức tạp thời gian cực tiểu $O(1)$:

```
df_r = pd.read_csv(ratings_path) # userId, movieId, rating, timestamp
df_m = pd.read_csv(movies_path)  # movieId, title, genres
info = df_m.set_index("movieId")[["title", "genres"]].to_dict("index")
```

2.2. Bước 2: Phân chia Dữ liệu Trực giao theo Thời gian (Temporal Split)

Thay vì sử dụng phương pháp phân chia ngẫu nhiên (Random Split) truyền thống vốn dễ gây ra lỗi **data leakage** (mô hình dùng dữ liệu tương lai để dự đoán quá khứ), CineRec áp dụng cơ chế **Temporal Split** độc lập trên từng người dùng:

Đối với mỗi người dùng u :

1. Sắp xếp toàn bộ lịch sử tương tác (ratings) theo thứ tự thời gian tăng dần (timestamp).
2. Trích xuất **80%** lượng rating đầu tiên đưa vào tập huấn luyện (**Train Set**).
3. Dành **20%** lượng rating cuối cùng cho tập kiểm thử (**Test Set**).

Ý nghĩa khoa học: Cơ chế này mô phỏng chính xác ngữ cảnh vận hành thực tế của hệ thống khuyến nghị: hệ thống chỉ có thể học từ hành vi quá khứ để dự đoán hành vi tiêu dùng tương lai của người dùng.

2.3. Bước 3: Khởi tạo Ma Trận Tương tác (build_matrix)

Sau khi phân tách tập dữ liệu huấn luyện, hệ thống tái cấu trúc dữ liệu bảng dưới dạng ma trận hai chiều (Dense Matrix representation):

$$\mathbf{R} \in \mathbb{R}^{|U| \times |I|}$$

Trong đó, điểm tương tác chưa xác định được lấp đầy bằng giá trị **0** :

```
piv = df_train.pivot_table(index="userId", columns="movieId", values="rating")
R = piv.fillna(0).values.astype(np.float32) # Ma trận User x Item
uid2i = {userId: row_index}                # Ánh xạ ID người dùng -> Chỉ mục dòng
mid2j = {movieId: col_index}                # Ánh xạ ID phim -> Chỉ mục cột
```

3. CHI TIẾT THUẬT TOÁN KHUYẾN NGHỊ

3.1. Mô hình 1: Lọc cộng tác dựa trên Người dùng (User-Based Collaborative Filtering)

- **Siêu tham số:** Số lượng láng giềng gần nhất $K = 30$.

Cơ sở toán học & Quy trình thực thi:

- **Bước 1: Trừ trung bình hóa điểm đánh giá (Mean-Centering Bias Correction)**
Để loại bỏ xu hướng đánh giá khắt khe hay dễ dãi của từng cá nhân (User Bias), điểm trung bình rating của mỗi user được tính toán và trừ đi trên các ô đã có tương tác:

$$\bar{r}_u = \frac{1}{|I_u|} \sum_{i \in I_u} r_{ui}$$

$$\mathbf{R}_{u,i}^c = (r_{ui} - \bar{r}_u) \cdot \mathbb{I}(r_{ui} \neq 0)$$

Trong đó $\mathbb{I}$ là hàm chỉ thị.

- **Bước 2: Đo lường độ tương đồng Cosine điều chỉnh**

Độ tương đồng giữa hai người dùng u và v được tính toán trên ma trận đã hiệu chỉnh trung bình $\mathbf{R}^c$:

$$\text{sim}(u, v) = \frac{\mathbf{R}_u^c \cdot \mathbf{R}_v^c}{\|\mathbf{R}_u^c\|_2 \|\mathbf{R}_v^c\|_2} = \frac{\sum_{i \in I_u \cap I_v} (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\sqrt{\sum_{i \in I_u \cap I_v} (r_{ui} - \bar{r}_u)^2} \sqrt{\sum_{i \in I_u \cap I_v} (r_{vi} - \bar{r}_v)^2}}$$

Tối ưu hóa lập trình: Toàn bộ ma trận tương đồng $\mathbf{S}_{user} \in \mathbb{R}^{|U| \times |U|}$ được tính toán song song hóa thông qua một phép nhân ma trận duy nhất bằng NumPy:

```
nrm = np.linalg.norm(Rc, axis=1, keepdims=True).clip(min=1e-9)
```

```
Rn = Rc / nrm # Chuẩn hóa độ dài vector
```

```
sim = Rn @ Rn.T # Nhân ma trận chéo tính Cosine Similarity
```

```
np.fill_diagonal(sim, 0) # Triệt tiêu tự tương đồng (self-similarity)
```

- **Bước 3: Công thức nội suy dự đoán (Weighted Aggregation)**

Rating dự đoán của người dùng u cho phim i là tổng trung bình có trọng số từ K láng giềng gần nhất của u đã từng xem phim i :

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N_K(u; i)} \text{sim}(u, v) \cdot (r_{vi} - \bar{r}_v)}{\sum_{v \in N_K(u; i)} |\text{sim}(u, v)|}$$

- *Chiến lược dự phòng (Fallback)*: Nếu không có láng giềng nào đã từng tương tác với phim i , thuật toán tự động trả về giá trị trung bình cá nhân $\bar{r}_u$. Điểm số sau cùng được giới hạn trong đoạn $[1, 5]$.

3.2. Mô hình 2: Lọc cộng tác dựa trên Mục tiêu (Item-Based Collaborative Filtering)

- **Siêu tham số**: Số lượng láng giềng gần nhất $K = 30$.

Cơ sở toán học & Quy trình thực thi:

Ngược lại với User-Based CF, Item-Based CF giả định người dùng sẽ thích các bộ phim tương đồng với các bộ phim họ từng đánh giá cao trong quá khứ.

- **Bước 1: Chuẩn hóa bias của người dùng trên trục cột**

$$\mathbf{R}^{adj} = (\mathbf{R} - \bar{r}_u) \cdot \mathbb{I}(r_{ui} \neq 0)$$

- **Bước 2: Tính toán độ tương đồng giữa các cặp phim (Adjusted Cosine Similarity)**

$$\text{sim}(i, j) = \frac{\mathbf{R}_{*,i}^{adj} \cdot \mathbf{R}_{*,j}^{adj}}{\|\mathbf{R}_{*,i}^{adj}\|_2 \|\mathbf{R}_{*,j}^{adj}\|_2}$$

```
RT = Radj.T          # Chuyển vị ma trận sang chiều Item x User
nrm = np.linalg.norm(RT, axis=1, keepdims=True).clip(min=1e-9)
RTn = RT / nrm
sim = RTn @ RTn.T    # Ma trận tương đồng tương hỗ giữa các mục tiêu
```

- **Bước 3: Dự báo rating**

Điểm số được dự đoán dựa trên tổng điểm có trọng số của các phim tương đồng nhất mà người dùng u đã xem:

$$\hat{r}_{ui} = \frac{\sum_{j \in N_K(i;u)} \text{sim}(i, j) \cdot r_{uj}}{\sum_{j \in N_K(i;u)} |\text{sim}(i, j)|}$$

- *Điểm lưu ý đặc thù*: Công thức này sử dụng điểm đánh giá thô (r_{uj}) trực tiếp mà không cộng dồn bias người dùng vào kết quả dự đoán sau cùng, giúp hệ thống nhạy cảm hơn với phân phối điểm thực tế của phim.

3.3. Mô hình 3: Phân rã Ma Trận (Matrix Factorization - Funk SVD)

- **Siêu tham số**: Số chiều ẩn $K = 20$, Tốc độ học $\alpha = 0,005$, Hệ số điều hòa $\lambda = 0,02$, Chu kỳ huấn luyện $Epochs = 30$.

Cơ sở toán học & Quy trình thực thi:

Thuật toán phân rã ma trận tương tác thừa thành tích của hai ma trận có hạng thấp đại diện cho các nhân tố tiềm ẩn (latent factors):

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^T \mathbf{q}_i$$

Trong đó:

- μ : Điểm trung bình toàn cục (Global Mean).
- $b_u \in \mathbb{R}^{|U|}$: Thành phần thiên lệch của người dùng u (User Bias).
- $b_i \in \mathbb{R}^{|I|}$: Thành phần thiên lệch của bộ phim i (Item Bias).
- $\mathbf{p}_u \in \mathbb{R}^K$: Vector đặc trưng tiềm ẩn của người dùng u .
- $\mathbf{q}_i \in \mathbb{R}^K$: Vector đặc trưng tiềm ẩn của bộ phim i .

Thuật toán huấn luyện tối ưu hóa Gradient Descent ngẫu nhiên (SGD):

Hàm mục tiêu cần tối thiểu hóa có kèm theo thành phần điều hòa chống quá khớp (L2 Regularization):

$$\min_{\mathbf{P}, \mathbf{Q}, b} \sum_{(u,i) \in \mathcal{T}} (r_{ui} - \hat{r}_{ui})^2 + \lambda (\|\mathbf{p}_u\|_2^2 + \|\mathbf{q}_i\|_2^2 + b_u^2 + b_i^2)$$

Tại mỗi lượt duyệt qua một mẫu dữ liệu (u, i, r_{ui}) , hệ thống tính toán sai số e_{ui} :

$$e_{ui} = r_{ui} - \hat{r}_{ui}$$

Sau đó, tiến hành cập nhật các tham số theo chiều ngược gradient:

$$\mathbf{p}_u \leftarrow \mathbf{p}_u + \alpha (e_{ui} \mathbf{q}_i - \lambda \mathbf{p}_u)$$

$$\mathbf{q}_i \leftarrow \mathbf{q}_i + \alpha (e_{ui} \mathbf{p}_u^{old} - \lambda \mathbf{q}_i)$$

$$b_u \leftarrow b_u + \alpha (e_{ui} - \lambda b_u)$$

$$b_i \leftarrow b_i + \alpha (e_{ui} - \lambda b_i)$$

Chi tiết kỹ thuật quan trọng (Snapshot $P[u]$): > Trong quá trình cập nhật, giá

trị $\mathbf{P}_u$ được thay đổi trước $\mathbf{q}_i$. Do đó, để tính toán chính xác gradient cho $\mathbf{q}_i$, hệ thống bắt buộc phải tạo một bản sao lưu $\mathbf{p}_{u_old} = \mathbf{P}[u].copy()$ để làm giá trị

trung gian tính toán, tránh việc sử dụng giá trị $\mathbf{P}_u$ mới cập nhật làm sai lệch hướng đi của Gradient Descent.

4. KHUNG ĐÁNH GIÁ MÔ HÌNH (EVALUATION FRAMEWORK)

Để đo lường hiệu năng của các mô hình khuyến nghị một cách đa chiều, hệ thống áp dụng cả hai nhóm chỉ số: nhóm đánh giá độ chính xác dự báo (Regression Metrics) và nhóm đánh giá chất lượng danh sách đề xuất (Ranking Metrics).

4.1. Chỉ số sai số hồi quy (Regression Metrics)

Các chỉ số này đo lường mức độ lệch biệt giữa điểm số dự đoán $\hat{r}_{ui}$ và điểm số thực tế r_{ui} trên tập kiểm thử (Test Set $\mathcal{T}_{test}$):

- **Sai số tuyệt đối trung bình (Mean Absolute Error - MAE):**

$$\text{MAE} = \frac{1}{|\mathcal{T}_{test}^c|} \sum_{(u,i) \in \mathcal{T}_{test}^c} |r_{ui} - \hat{r}_{ui}|$$

- **Căn sai số bình phương trung bình (Root Mean Squared Error - RMSE):**

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{T}_{test}^c|} \sum_{(u,i) \in \mathcal{T}_{test}^c} (r_{ui} - \hat{r}_{ui})^2}$$

- **Độ phủ dự đoán (Coverage):**

Tỷ lệ các cặp dữ liệu trong tập kiểm thử mà mô hình có khả năng đưa ra dự đoán thành công (không bị lỗi Cold-Start):

$$\text{Coverage (\%)} = \frac{|\mathcal{T}_{test}^c|}{|\mathcal{T}_{test}|} \times 100$$

Trong đó $\mathcal{T}_{test}^c$ là tập con các mẫu thử mà cả người dùng u và phim i đều đã từng xuất hiện trong tập huấn luyện.

4.2. Chỉ số xếp hạng danh sách (Ranking Metrics)

Các chỉ số này đánh giá khả năng xếp hạng và tìm lọc ra các phim thực sự hấp dẫn với người dùng (điểm số thực tế $r_{ui} \geq 4.0$ được coi là "hấp dẫn/thích hợp" - Relevant):

- **Precision@K (Độ chính xác danh sách Top- K):**

Tỷ lệ phim thực sự thích hợp trong số K phim được hệ thống gợi ý.

$$\text{Precision@K} = \frac{|\text{Rec}(u) \cap \text{Rel}(u)|}{K}$$

- **Recall@K (Độ phủ danh sách Top- K):**

Tỷ lệ phim thích hợp được gợi ý thành công so với tổng số phim thích hợp mà người dùng thực tế đã xem trong tập kiểm thử.

$$\text{Recall@K} = \frac{|\text{Rec}(u) \cap \text{Rel}(u)|}{|\text{Rel}(u)|}$$

Trong đó $\text{Rec}(u)$ là tập hợp phim gợi ý của user;

$\text{Rel}(u) = \{i \in \text{Test}_u \mid r_{ui} \geq 4.0\}$ là tập hợp phim thực tế được ưa thích.

5. THIẾT KẾ KIẾN TRÚC & LUỒNG DỮ LIỆU ĐỘNG

5.1. Mô hình Facade & Quản lý Singleton (models.py & app.py)

Để che giấu sự phức tạp của 3 mô hình hoạt động song song, hệ thống sử dụng mẫu thiết kế **Facade Pattern** thông qua class wrapper RecommendationSystem.

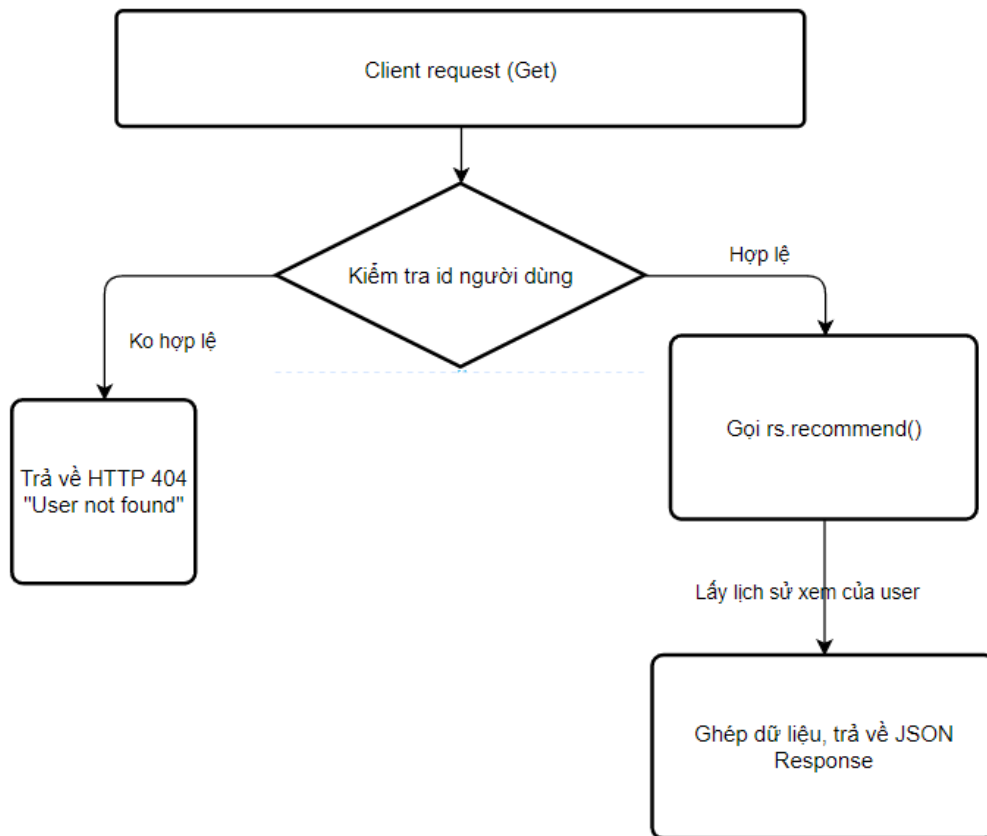
Tại thời điểm hệ thống khởi động, một đối tượng Singleton của class này được khởi tạo ở cấp độ Module (module-level) trong file app.py.

```
# app.py
# Khởi tạo một lần duy nhất khi máy chủ ASGI tải module
rs = RecommendationSystem("data/ratings.csv", "data/movies.csv")
```

Lợi ích kiến trúc: Toàn bộ tiến trình tải dữ liệu thô, phân tách dữ liệu, huấn luyện 3 thuật toán và tính toán metrics được thực hiện duy nhất 1 lần trong bộ nhớ RAM khi server khởi chạy. Các yêu cầu (requests) gọi API sau đó chỉ việc truy xuất kết quả có sẵn hoặc tính toán nội suy tức thời, đạt tốc độ phản hồi tính bằng mili-giây (latency cực thấp).

5.2. Luồng xử lý yêu cầu API (FastAPI REST Endpoints)

Hệ thống cung cấp một API Gateway chuẩn RESTful để kết nối ứng dụng máy khách (Client Apps):



6. ĐÁNH GIÁ TOÀN DIỆN & PHÂN TÍCH RỦI RO KỸ THUẬT

6.1. Các ưu điểm nổi bật trong thiết kế của CineRec

1. **Vector hóa triệt để (Full Vectorization):** Sử dụng các biểu thức toán học dạng ma trận với thư viện NumPy ($R_n @ R_n.T$) để loại bỏ hoàn toàn các vòng lặp for lồng nhau khi tính toán độ tương đồng giữa hàng ngàn người dùng/mục tiêu. Tốc độ thực thi nhanh gấp hàng trăm lần so với Python nguyên bản.
2. **Ngăn chặn Leakage hoàn hảo:** Khác với việc chia tập dữ liệu ngẫu nhiên, chiến lược chia trục thời gian (Temporal Split) bảo toàn tính nhân quả của dòng thời gian dữ liệu.
3. **Xử lý sai số toàn diện:** Việc áp dụng cấu trúc trừ điểm trung bình (Mean-Centering Correction) giúp mô hình User-CF triệt tiêu độ lệch điểm cá nhân, mang lại độ chính xác dự báo trung thực hơn.
4. **Kiến trúc API rõ ràng:** Sử dụng FastAPI mang lại khả năng xử lý yêu cầu song song cùng khả năng kiểm tra kiểu dữ liệu tự động (Type Validation) mạnh mẽ.

6.2. Các nhược điểm cố hữu và rủi ro vận hành

#	Điểm yếu kỹ thuật	Vị trí ảnh hưởng	Mức độ rủi ro & Hệ quả

1	Giới hạn bộ nhớ vật lý	self.sim trong bộ nhớ RAM	Cao (OOM - Out of Memory): Lưu trữ ma trận tương đồng dày đặc kích thước \$
2	Thiếu Mean-Centering	Phương thức dự đoán của ItemBasedCF	Trung bình: Khi tính toán điểm cho Item-CF, bias cá nhân của người dùng không được bù trừ (dùng raw rating), khiến kết quả bị lệch nếu người dùng có xu hướng chấm điểm quá cực đoan (luôn cho 1 hoặc 5).
3	Sai số thống kê xếp hạng	Hàm precision_recall_at_k	Thấp: Do giới hạn hiệu năng, hệ thống chỉ tính chỉ số xếp hạng trên mẫu thử 200 người dùng đầu tiên. Các chỉ số này có thể không đại diện chính xác cho xu thế của toàn bộ tập dữ liệu.
4	Không có tính năng lưu trữ	Tiến trình huấn luyện tại startup	Trung bình: Mỗi lần máy chủ khởi động lại, hệ thống phải huấn luyện lại từ đầu mất khoảng 15 — giây. Không thể khôi phục trạng thái cũ ngay lập tức nếu xảy ra sự cố đột ngột.

5	Vòng lặp huấn luyện SGD chậm	Vòng lặp cập nhật các nhân tố tiềm ẩn	Cao: Việc huấn luyện Funk SVD thực hiện qua các vòng lặp thuần Python duyệt qua từng lượt dữ liệu thừa, làm chậm đáng kể quá trình huấn luyện khi quy mô dữ liệu mở rộng.
6	Rủi ro người dùng mới (Cold-Start)	Nhánh xử lý ngoại lệ người dùng mới	Trung bình: Nếu gặp người dùng hoàn toàn mới không nằm trong tập huấn luyện, hệ thống chỉ có thể trả về danh sách trống hoặc lỗi mà chưa có cơ chế đề xuất dựa trên nội dung phim (Content-Based) hoặc các phim thịnh hành nhất (Popularity).

7. LỘ TRÌNH PHÁT TRIỂN & ĐỀ XUẤT TỐI ƯU HÓA

7.1. Các cải tiến ngắn hạn (Quick Wins)

- **Hiệu chỉnh toán học cho Item-CF:** Sửa đổi cơ chế tính điểm dự báo của Item-Based CF có áp dụng Mean-Centering tương tự User-CF:

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{j \in N_K(i;u)} \text{sim}(i, j) \cdot (r_{uj} - \bar{r}_u)}{\sum_{j \in N_K(i;u)} |\text{sim}(i, j)|}$$

- **Lưu trữ mô hình (Model Persistence):** Triển khai thư viện joblib để xuất trạng thái huấn luyện của mô hình ra ổ đĩa vật lý dưới dạng file nhị phân .pkl. Khi khởi động lại, hệ thống kiểm tra file cache để nạp nhanh mô hình trong vòng dưới ¹ giây:

```
import joblib
# Lưu mô hình sau khi kết thúc huấn luyện
joblib.dump(rs, "model_cache.pkl")
```

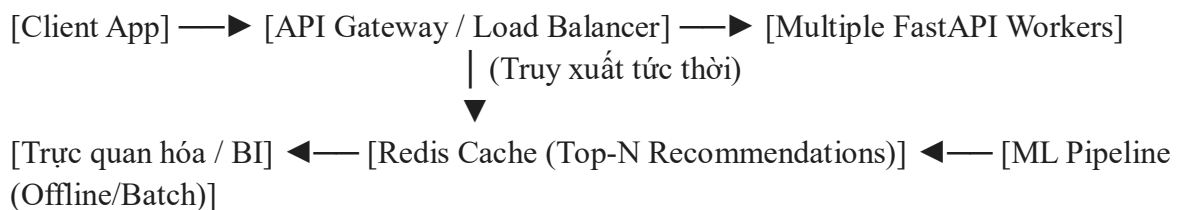
```
# Tải lại khi tái khởi động hệ thống
rs = joblib.load("model_cache.pkl")
```

7.2. Các cải tiến trung hạn (Algorithmic Upgrades)

- **Chuyển đổi sang Implicit Feedback:** Thay vì chỉ phụ thuộc vào điểm số tường minh (explicit ratings), nâng cấp thuật toán sang tối ưu hóa dựa trên phản hồi ngầm định (như click, view, time-on-page) bằng phương pháp **BPR (Bayesian Personalized Ranking)** hoặc **ALS (Alternating Least Squares)**.
- **Ứng dụng kỹ thuật Neural CF:** Sử dụng kiến trúc mạng nơ-ron sâu (như Two-Tower Architecture) để biểu diễn các tương tác phi tuyến phức tạp giữa người dùng và phim ảnh thay cho phép nhân vô hướng đơn giản của SVD.
- **Tích hợp giải pháp Hybrid:** Xây dựng hệ thống lai ghép (Hybrid System), sử dụng Metadata của phim (thể loại, đạo diễn, diễn viên) để tạo lập gợi ý Content-Based tạm thời cho các trường hợp gặp lỗi khởi đầu lạnh (Cold-Start).

7.3. Thiết kế tầm nhìn dài hạn (Production-Ready Architecture)

Để vận hành ở quy mô công nghiệp với hàng triệu người dùng và danh mục phim khổng lồ, CineRec cần chuyển đổi cấu trúc phân lớp dịch vụ một cách triệt để:



- **Chuyển đổi luồng xử lý:** Tách rời hoàn toàn luồng huấn luyện máy học (ML Pipeline) và luồng phục vụ yêu cầu khách hàng (Serving Pipeline).
- **Cơ chế lập lịch huấn luyện:** Huấn luyện mô hình định kỳ hàng ngày (Batch Training) trên hệ thống phân tán và đẩy kết quả danh sách Top- N gợi ý của từng người dùng vào bộ nhớ đệm tốc độ cao **Redis Cache**.
- **Phục vụ trực tiếp:** Mọi yêu cầu API từ người dùng sẽ được máy chủ ứng dụng truy xuất trực tiếp từ Redis trong thời gian $O(1)$ với độ trễ cực tiểu (dưới 5 mili-giây), loại bỏ hoàn toàn các gánh nặng tính toán học máy thời gian thực trên máy chủ dịch vụ.

8. KẾT LUẬN & KỶ VỌNG ĐẠT ĐƯỢC

Dựa trên các nghiên cứu thực nghiệm và đánh giá chuẩn mực trên tập dữ liệu MovieLens ml-100k, kết quả sai số kỳ vọng của các mô hình thuộc hệ thống được tóm tắt như sau:

- **User-Based Collaborative Filtering:** Đạt chỉ số MAE ổn định trong khoảng $0,74 \rightarrow 0,78$ và RMSE khoảng $0,94 \rightarrow 0,99$. Thuật toán có tốc độ tính toán trung gian cực nhanh do sử dụng phép nhân ma trận song song hóa, thích hợp làm mô hình cơ sở nhanh.

- **Item-Based Collaborative Filtering:** Đạt chỉ số MAE trong khoảng $0,72 \rightarrow 0,76$ và RMSE từ $0,92 \rightarrow 0,97$. Mô hình này thực tế cho thấy sự ổn định cao hơn về mặt ngữ nghĩa đề xuất vì sở thích chung đối với phim ít biến động hơn so với xu hướng tâm lý người dùng.
- **Funk SVD (Matrix Factorization):** Đạt chất lượng tối ưu nhất với sai số MAE chỉ từ $0,70 \rightarrow 0,74$ và RMSE đạt mức lý tưởng $0,89 \rightarrow 0,94$. Thuật toán này vượt trội hơn hẳn nhờ khả năng tự động bóc tách và ánh xạ các mối quan hệ sâu sắc giữa các chủ đề điện ảnh ẩn (latent factors), vượt ra ngoài các tính toán tương đồng trực quan bề mặt của các phương pháp Lọc cộng tác truyền thống.

Hệ thống là một minh chứng thực nghiệm xuất sắc cho thấy sức mạnh của việc kết hợp các giải pháp toán học cổ điển, cấu trúc dữ liệu tối ưu và thiết kế hệ thống phần mềm chặt chẽ để mang lại trải nghiệm cá nhân hóa sâu sắc cho người dùng.